

# Dinamic

## Dice Combinations

---

```
#include

using namespace std;
const int maxN = 1e6;
const int MOD = 1e9+7;

int N, dp[maxN+1];

int main(){
    scanf("%d", &N);

    dp[0] = 1;
    for(int i = 1; i <= N; i++)
        for(int j = 1; j <= 6 && i-j >= 0; j++)
            dp[i] = (dp[i] + dp[i-j]) % MOD;

    printf("%d\n", dp[N]);
}
```

---

## Minimizing Coins

---

```
#include

using namespace std;
const int maxX = 1e6;
const int INF = 0x3f3f3f3f;

int N, X, c, dp[maxX+1];

int main(){
    scanf("%d %d", &N, &X);
    fill(dp+1, dp+X+1, INF);
    for(int i = 0; i < N; i++){
        scanf("%d", &c);
        for(int j = 0; j <= X-c; j++){
            if(dp[j] != INF)
                dp[j+c] = min(dp[j+c], dp[j]+1);
        }
        printf("%d\n", dp[X] == INF ? -1 : dp[X]);
    }
}
```

---

## Coin Combinations I

---

```
#include

using namespace std;
typedef long long ll;
const int maxN = 100;
const int maxX = 1e6;
const ll MOD = 1e9+7;

int N, X, c[maxN];
ll dp[maxX+1];

int main(){
    scanf("%d %d", &N, &X);
    for(int i = 0; i < N; i++)
        scanf("%d", &c[i]);

    dp[0] = 1;
    for(int i = 0; i < X; i++)
        if(dp[i] != 0)
            for(int j = 0; j < N; j++)
                if(i+c[j] <= X)
                    dp[i+c[j]] = (dp[i+c[j]] + dp[i]) % MOD;

    printf("%lld\n", dp[X]);
}
```

---

## Coin Combinations II

```
#include

using namespace std;
typedef long long ll;
const int maxX = 1e6;
const ll MOD = 1e9+7;

int N, X, c;
ll dp[maxX+1];

int main(){
    scanf("%d %d", &N, &X);

    dp[0] = 1;
    for(int i = 0; i < N; i++){
        scanf("%d", &c);
        for(int j = 0; j <= X-c; j++)
            dp[j+c] = (dp[j+c] + dp[j]) % MOD;
    }

    printf("%lld\n", dp[X]);
}
```

---

## Removing Digits

```
#include

using namespace std;
const int maxN = 1e6;
const int INF = 0x3f3f3f3f;

int N, dp[maxN+1];

int main(){
    scanf("%d", &N);
    fill(dp+1, dp+N+1, INF);

    for(int i = 1; i <= N; i++){
        int d = i;
        while(d > 0){
            if(d%10 != 0)
                dp[i] = min(dp[i], dp[i-(d%10)]+1);
            d /= 10;
        }
    }

    printf("%d\n", dp[N]);
}
```

## Grid Paths I

```
#include

using namespace std;
typedef long long ll;
const int maxN = 1000;
const ll MOD = 1e9+7;

int N;
char c[maxN+1][maxN+1];
ll dp[maxN+1][maxN+1];

int main(){
    scanf("%d", &N);
    for(int i = 1; i <= N; i++)
        for(int j = 1; j <= N; j++)
            scanf(" %c", &c[i][j]);

    dp[1][1] = (c[1][1] == '.');
    for(int i = 1; i <= N; i++){
        for(int j = 1; j <= N; j++){
            if(c[i][j] == '.'){
                if(c[i-1][j] == '.') dp[i][j] += dp[i-1][j];
                if(c[i][j-1] == '.') dp[i][j] += dp[i][j-1];
                dp[i][j] %= MOD;
            }
        }
    }

    printf("%lld\n", dp[N][N]);
}
```

# Book Shop

```
#include

using namespace std;
const int maxN = 1000;
const int maxX = 1e5;

int N, X, h[maxN], s[maxN], dp[maxX+1];

int main(){
    scanf("%d %d", &N, &X);
    for(int i = 0; i < N; i++) scanf("%d", &h[i]);
    for(int i = 0; i < N; i++) scanf("%d", &s[i]);

    fill(dp+1, dp+X+1, -1);
    for(int i = 0; i < N; i++)
        for(int j = X-h[i]; j >= 0; j--)
            if(dp[j] != -1)
                dp[j+h[i]] = max(dp[j+h[i]], dp[j]+s[i]);

    for(int i = 1; i <= X; i++)
        dp[i] = max(dp[i], dp[i-1]);
    printf("%d\n", dp[X]);
}
```

## Array Description

```
#include

using namespace std;
typedef long long ll;
const int maxN = 1e5;
const int maxM = 100;
const ll MOD = 1e9+7;

int N, M, x[maxN+1];
ll ans, dp[maxN+1][maxM+1];

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 1; i <= N; i++)
        scanf("%d", &x[i]);

    if(x[1])
        dp[1][x[1]] = 1;
    else
        for(int i = 1; i <= M; i++)
            dp[1][i] = 1;

    for(int i = 2; i <= N; i++){
        for(int j = 1; j <= M; j++){
            dp[i][j] = dp[i-1][j];
            if(j != 1) dp[i][j] += dp[i-1][j-1];
            if(j != M) dp[i][j] += dp[i-1][j+1];
            dp[i][j] %= MOD;
        }

        if(x[i])
```

```

        for(int j = 0; j <= M; j++)
            if(j != x[i])
                dp[i][j] = 0;
    }

    for(int i= 1; i <= M; i++)
        ans = (ans + dp[N][i]) % MOD;
    printf("%lld\n", ans);
}

```

## Counting Towers

```

#include

using namespace std;
typedef long long ll;
const int maxN = 1e6;
const ll MOD = 1e9+7;

int T, N;
ll dp[maxN+1];

int main(){
    dp[1] = 2;
    dp[2] = 8;
    for(int i = 3; i <= maxN; i++)
        dp[i] = ((6*dp[i-1] - 7*dp[i-2]) % MOD + MOD) % MOD;

    scanf("%d", &T);
    for(int t = 0; t < T; t++){
        scanf("%d", &N);
        printf("%lld\n", dp[N]);
    }
}

```

## Edit Distance

```

#include

using namespace std;
const int maxN = 5e3+5;

int N, M, dp[maxN][maxN];
char a[maxN], b[maxN];

int main(){
    scanf("%s %s", a, b);
    N = (int) strlen(a);
    M = (int) strlen(b);

    memset(dp, 0x3f, sizeof(dp));
    for(int i = 0; i <= N; i++){

```

```

    for(int j = 0; j <= M; j++){
        if(i == 0) dp[i][j] = j;
        else if(j == 0) dp[i][j] = i;
        else if(a[i-1] == b[j-1]) dp[i][j] = dp[i-1][j-1];
        else dp[i][j] = 1 + min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]});
    }
}

printf("%d\n", dp[N][M]);
}

```

## Longest Common Subsequence

```

#include

using namespace std;
const int maxN = 1e3+1;

int N, M, a[maxN], b[maxN];
int len[maxN][maxN];
char ptr[maxN][maxN];

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 1; i <= N; i++) scanf("%d", &a[i]);
    for(int i = 1; i <= M; i++) scanf("%d", &b[i]);

    for(int i = 1; i <= N; i++){
        for(int j = 1; j <= M; j++){
            if(len[i][j] <= len[i-1][j]){
                len[i][j] = len[i-1][j];
                ptr[i][j] = 'L';
            }
            if(len[i][j] <= len[i][j-1]){
                len[i][j] = len[i][j-1];
                ptr[i][j] = 'U';
            }
            if(a[i] == b[j] && len[i][j] <= len[i-1][j-1] + 1){
                len[i][j] = len[i-1][j-1] + 1;
                ptr[i][j] = 'D';
            }
        }
    }

    vector ans;
    int i = N, j = M;
    while(i != 0 && j != 0){
        char dir = ptr[i][j];

        if(dir == 'L') i--;
        else if(dir == 'U') j--;
        else if(dir == 'D'){
            ans.push_back(a[i]);
            i--; j--;
        }
    }
    reverse(ans.begin(), ans.end());

    int l = len[N][M];
    printf("%d\n", l);
    for(int i = 0; i < l; i++)

```

```
        printf("%d%c", ans[i], (" \n")[i==l-1]);  
    }
```

---

## Rectangle Cutting

```
#include  
  
using namespace std;  
const int INF = 0x3f3f3f3f;  
  
int A, B, dp[501][501];  
  
int solve(int a, int b){  
    if(a > b) swap(a, b);  
    if(dp[a][b] != INF) return dp[a][b];  
    if(a == b) return dp[a][b] = 0;  
    if(a == 1 || b == 1) return dp[a][b] = (a == 1 ? b-1 : a-1);  
    for(int i = 1; i < a; i++)  
        dp[a][b] = min(dp[a][b], solve(i, b) + solve(a-i, b) + 1);  
    for(int i = 1; i < b; i++)  
        dp[a][b] = min(dp[a][b], solve(a, i) + solve(a, b-i) + 1);  
    return dp[a][b];  
}  
  
int main(){  
    memset(dp, 0x3f, sizeof(dp));  
    scanf("%d %d", &A, &B);  
    printf("%d\n", solve(A, B));  
}
```

---

## Money Sums

```
#include  
  
using namespace std;  
const int maxX = 1e5;  
  
int N, x, cnt, largest;  
bool dp[maxX+1];  
  
int main(){  
    scanf("%d", &N);  
  
    dp[0] = true;  
    for(int i = 0; i < N; i++){  
        scanf("%d", &x);  
        for(int j = maxX-x; j >= 0; j--)  
            if(dp[j])  
                dp[j+x] = true;  
    }  
  
    for(int i = 1; i <= maxX; i++){
```

```

        if(dp[i]){
            largest = i;
            cnt++;
        }
    }

    printf("%d\n", cnt++);
    for(int i = 1; i <= maxX; i++)
        if(dp[i])
            printf("%d%c", i, (" \n")[i==largest]);
}

```

## Removal Game

```

#include

using namespace std;
typedef long long ll;
const int maxN = 5000;

int N, x[maxN+1];
ll p[maxN+1], dp[maxN+1][maxN+1];
bool found[maxN+1][maxN+1];

ll sum(int l, int r){
    return p[r] - p[l-1];
}

ll solve(int l, int r){
    if(found[l][r]) return dp[l][r];
    if(l == r) return x[l];
    found[l][r]=true;
    return dp[l][r] = max(x[l]+sum(l+1, r)-solve(l+1, r), x[r]+sum(l,r-1)-solve(l,r-1));
}

int main(){
    scanf("%d", &N);
    for(int i = 1; i <= N; i++){
        scanf("%d", &x[i]);
        p[i] = p[i-1] + x[i];
    }
    printf("%lld\n", solve(1, N));
}

```

## Two Sets II

```

#include
using namespace std;

long long mod = 1000000007;
long long dp[70001];

```

```

int main() {
    int n;
    cin >> n;
    long long sum = (1ll * n * (n + 1)) / 2;
    if (sum % 2) {
        cout << 0 << endl;
        return 0;
    }
    long long target = sum / 2;

    dp[0] = 1;
    // i= i; j-- ) {
        dp[j] += dp[j - i];
        dp[j] %= mod;
    }
}
cout << dp[target] << endl;
return 0;
}

```

## Mountain Range

```

#include

using namespace std;
typedef pair pii;
const int maxN = 2e5+1;
const int INF = 0x3f3f3f3f;

int N, dp[maxN];
vector tallers;
vector mountains;
set< S_l;
set S_r;

int main(){
    scanf("%d", &N);

    for(int i = 1, h; i <= N; i++){
        scanf("%d", &h);
        mountains.push_back({h, i});
    }
    sort(mountains.begin(), mountains.end(), greater());

    int last_height = INF;
    for(pii m : mountains){
        int h = m.first, i = m.second;

        if(last_height != h){
            for(int t : tallers){
                S_l.insert(t);
                S_r.insert(t);
            }
            tallers.clear();
        }

        auto l_ptr = S_l.lower_bound(i);
        auto r_ptr = S_r.lower_bound(i);
        int l = (l_ptr == S_l.end() ? 0 : *l_ptr);
        int r = (r_ptr == S_r.end() ? 0 : *r_ptr);
        dp[i] = max(dp[l], dp[r]) + 1;
    }
}

```

```

        tallers.push_back(i);
        last_height = h;
    }

    int best = 0;
    for(int i = 1; i <= N; i++)
        best = max(best, dp[i]);
    printf("%d\n", best);
}

```

## Increasing Subsequence

```

#include

using namespace std;

int N, a;
multiset S;
multiset::iterator it;

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        scanf("%d", &a);
        S.insert(a);
        it = S.lower_bound(a);
        it++;
        if(it != S.end())
            S.erase(it);
    }
    printf("%d\n", (int) S.size());
}

```

## Projects

```

#include

using namespace std;
typedef long long ll;
typedef pair pii;
const int maxN = 2e5+1;

int N, a[maxN], b[maxN];
ll p[maxN], dp[2*maxN];
struct project {int time, id, type;} times[2*maxN];
map mp;

int main(){
    scanf("%d", &N);
    for(int i = 1; i <= N; i++){
        scanf("%d %d %lld", &a[i], &b[i], &p[i]);
    }
}

```

```

        times[2*i] = {a[i], i, 0};
        times[2*i+1] = {b[i], i, 1};
    }

    sort(times+2, times+2*N+2, [](project A, project B){
        return A.time == B.time ? A.id < B.id : A.time < B.time;
    });

    for(int i = 2; i <= 2*N+1; i++)
        if(!mp[times[i].time])
            mp[times[i].time] = i;

    for(int i = 2; i <= 2*N+1; i++){
        if(times[i].type == 0) dp[i] = dp[i-1];
        else dp[i] = max(dp[i-1], dp[mp[a[times[i].id]]-1] + p[times[i].id]);
    }
    printf("%lld\n", dp[2*N+1]);
}

```

## Elevator Rides

```

#include

using namespace std;
typedef pair pii;
typedef long long ll;
const int maxN = 20;
const int INF = maxN+1;

int N;
pii dp[1<

```

## Counting Tilings

```

#include

using namespace std;
typedef long long ll;
const int maxN = 11;
const int maxM = 1001;
const ll MOD = 1e9+7;

int N, M;
ll dp[maxN][maxM][1<

```

# Counting Numbers

```
#include

using namespace std;
typedef long long ll;
const int maxN = 20;

bool tight[maxN];
ll dp[10][maxN];

ll solve(ll x){
    if(x <= 10) return x;

    vector D;
    memset(dp, 0, sizeof(dp));
    memset(tight, 0, sizeof(tight));
    for(int i = 0; x; i++){
        D.push_back(x%10);
        x /= 10;
    }
    reverse(D.begin(), D.end());
    int N = (int) D.size();

    tight[0] = true;
    for(int i = 1; i < N; i++){
        tight[i] = tight[i-1] && (D[i] != D[i-1]);
    }
    for(int d = 1; d < D[0]; d++){
        dp[d][0] = 1;
    }

    for(int i = 1; i < N; i++){
        // Prefix [0..i-1] is same as X
        if(tight[i-1])
            for(int d = 0; d < D[i]; d++){
                if(d != D[i-1])
                    dp[d][i]++;
            }

        // Prefix [0..i-1] is all leading zeros
        for(int d = 1; d <= 9; d++){
            dp[d][i]++;
        }

        // All other cases
        for(int d1 = 0; d1 <= 9; d1++){
            for(int d2 = 0; d2 <= 9; d2++){
                if(d1 != d2)
                    dp[d2][i] += dp[d1][i-1];
            }
        }

        ll cnt = tight[N-1];
        for(int d = 0; d <= 9; d++){
            cnt += dp[d][N-1];
        }
        return cnt;
    }

    int main(){
        ll a, b;
        scanf("%lld %lld", &a, &b);
        printf("%lld\n", solve(b) - solve(a-1));
    }
}
```

# Increasing Subsequence II

```
#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+5;
const ll MOD = 1e9+7;

int N;
ll ans, ds[maxN];
struct Operation { int x, idx; } ops[maxN];

void update(int idx, ll val){
    for(int i = idx; i < maxN; i += -i&i)
        ds[i] = (ds[i] + val) % MOD;
}

ll query(int idx){
    ll sum = 0;
    for(int i = idx; i > 0; i -= -i&i)
        sum = (sum + ds[i]) % MOD;
    return sum;
}

int main(){
    scanf("%d", &N);
    for(int i = 0, x; i < N; i++){
        scanf("%d", &x);
        ops[i] = {x, i+1};
    }
    sort(ops, ops+N, [](Operation A, Operation B){
        return A.x == B.x ? B.idx < A.idx : A.x < B.x;
    });

    for(int i = 0; i < N; i++){
        int idx = ops[i].idx;
        ll amnt = query(idx)+1;
        ans = (ans + amnt) % MOD;
        update(idx, amnt);
    }
    printf("%lld\n", ans);
}
```